



NGPU-LM: GPU-Accelerated N-Gram Language Model for Context-Biasing in Greedy ASR Decoding

Vladimir Bataev¹, Andrei Andrusenko¹, Lilit Grigoryan¹, Aleksandr Laptev², Vitaly Lavrukhin²,
Boris Ginsburg²

¹NVIDIA, Armenia; ²NVIDIA, USA

{vbataev, aandrusenko, lgrigoryan, alaptev, vlavrukhin}@nvidia.com

Abstract

Statistical n-gram language models are widely used for context-biasing tasks in Automatic Speech Recognition (ASR). However, existing implementations lack computational efficiency due to poor parallelization, making context-biasing less appealing for industrial use. This work rethinks data structures for statistical n-gram language models to enable fast and parallel operations for GPU-optimized inference. Our approach, named NGPU-LM, introduces customizable greedy decoding for all major ASR model types - including transducers, attention encoder-decoder models, and CTC - with less than 7% computational overhead. The proposed approach can eliminate more than 50% of the accuracy gap between greedy and beam search for out-of-domain scenarios while avoiding significant slowdown caused by beam search. The implementation of the proposed NGPU-LM will be open-sourced.

Index Terms: speech recognition, n-gram language model, context-biasing, greedy decoding

1. Introduction

The emergence of end-to-end approaches has significantly simplified the development of Automatic Speech Recognition (ASR) systems. The first end-to-end ASR model, based on Connectionist Temporal Classification (CTC) [1], allowed to learn the alignment between input acoustic features and target transcriptions without the time-consuming GMM-HMM training process used in conventional hybrid ASR systems [2]. Later, the introduction of the Recurrent Neural Transducer (RNN-T) [3], which incorporates an LSTM-based decoder module, enabled the model to better capture language dependencies. Advances in machine translation have further paved the way for adapting the Attention Encoder-Decoder (AED) approach [4] to speech recognition, thereby improving recognition accuracy and integrating additional modalities (e.g., translation, time-stems prediction, etc. [5, 6]) into a single model.

Recently, Large Language Models have been applied to speech tasks (SpeechLM) [7, 8], showing promise as the next evolution in speech foundation models. However, training and deploying SpeechLM models remain highly resource-intensive, less stable, and still relatively underexplored.

End-to-end ASR models often exhibit a bias toward the training data domain, which leads to accuracy degradation when encountering out-of-domain test data. To mitigate this, context-biasing techniques are used to incorporate additional out-of-domain information through either deep or shallow fusion. Deep fusion involves feeding context data into the ASR model during training or fine-tuning [9, 10], but this process can be computationally expensive. Similarly, SpeechLM supports context-biasing through prompting [11]. However, this

requires modifications to the training process (in-context learning) and does not scale well with an increasing number of target phrases. In contrast, shallow fusion-based methods [12, 13, 14] apply context biasing exclusively during the decoding stage, eliminating the need for costly model retraining.

The most common shallow fusion strategy is beam search decoding with an external language model (LM) [12]. While neural LMs based on Long Short-Term Memory (LSTM) [15] or Transformer [16] architectures can serve as the external model, they require a substantial amount of out-of-domain text for training. In contrast, a statistical LM based on n-grams offers a simpler alternative, requiring significantly fewer resources and a smaller training dataset. One popular implementation of such a model is provided by the open-source KenLM library [17], which represents the n-gram LM using a trie-based data structure optimized for CPU execution.

Although shallow fusion with an external LM improves out-of-domain recognition accuracy, beam search decoding introduces a significant slowdown compared to greedy decoding. This issue is particularly evident in Transducer-based ASR models, where beam search necessitates multiple evaluations of the Joint and Prediction network components. Moreover, the CPU-based implementation of the n-gram LM further restricts the decoding speed for other ASR architectures. Consequently, practitioners often face the need to choose between speed and accuracy in speech recognition.

Recent developments have introduced a more efficient greedy decoding method for Transducers based on the label-looping algorithm [18] coupled with CUDA graphs [19]. This advancement has significantly accelerated standard greedy decoding, widening the speed gap between the new GPU-based greedy approach and standard beam search.

In this paper, we present NGPU-LM, a novel GPU-accelerated implementation of a statistical n-gram language model. Our approach leverages a universal trie-based data structure that enables fast batched queries, thereby unlocking new possibilities for ASR decoding. NGPU-LM facilitates context-biasing in rapid greedy decoding on GPU across all major ASR architectures, including CTC, Transducers, and AED. The proposed method provides up to 10.6% relative WER improvement while avoiding the substantial slowdown typically associated with beam search.

The main contribution of our work is of two parts:

- NGPU-LM: A novel data structure representing a statistical n-gram language model optimized for parallel computing;
- Shallow Fusion in Greedy Decoding: Enabling fast context-biased decoding across major ASR architectures (CTC, Transducers, AED) with extremely low overhead.

Our implementation is open-sourced in the NeMo [20] toolkit.

N-Gram LM ARPA Format:

```

\1-grams:
-1.15 <unk> 0
0 <s> -0.30
-0.85 </s> 0
-0.67 the -0.30
-0.85 cat -0.30
-0.85 sat -0.30
-0.85 on -0.30
-0.85 mat -0.30

\2-grams:
-0.24 mat </s> 0
-0.22 <s> the -0.30
-0.22 on the -0.30
-0.49 the cat -0.30
-0.24 cat sat -0.30
-0.24 sat on -0.30
-0.49 the mat -0.30

\3-grams:
-0.10 the mat </s>
-0.09 sat on the
-0.18 <s> the cat
-0.10 the cat sat
-0.10 cat sat on
-0.18 on the mat

```

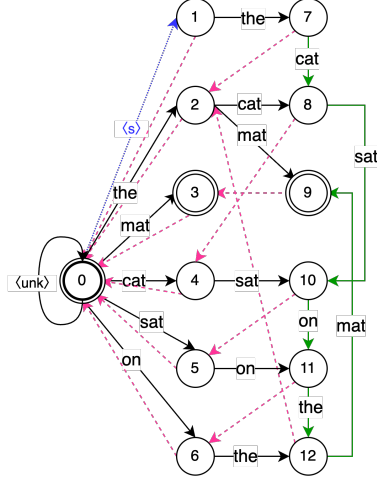


Figure 1: **3-gram LM** built from the text “the cat sat on the mat”. **Arrows: black** – unigram and bigram transitions; **green** – 3-gram (highest order) transitions; **pink dashed** – backoff transitions. **Double circles: final states.**

2. Methods

2.1. Statistical LM Background

N-gram LM Structure. Statistical N-gram language model is a probabilistic model used to predict the likelihood of a sequence of text tokens (word- or subword-level). It is based on frequencies of subsequences (n-grams) observed in textual training data. To explain the structure in depth, let’s consider the word-level n-gram LM built from text and stored in a human-readable ARPA format, see Fig. 1. Each line in an ARPA file describing the n-gram follows the format: “W(token|context) [context] token B(context + token)”, where context is a sequence of $N - 1$ tokens (context can be empty if $N = 1$). Probabilities (weights W) are stored as logarithms, allowing all calculations to be performed using summation instead of multiplication. Backoff weights B are used for discounting probabilities when transitioning to a smaller context. In addition to regular tokens, n-gram models include special symbols such as start-of-sequence $\langle s \rangle$, end-of-sequence $\langle /s \rangle$, and the unknown unigram $\langle unk \rangle$.

To score the sentence, the algorithm starts with the start-of-sequence token as a context and, iteratively, for each token, accumulates the log-probabilities for $W(\text{token}|\text{context})$. If the model does not contain $W(\text{token}|\text{context})$ explicitly, the algorithm iteratively reduces the context, going through backoffs until $W(\text{token}|\text{context}_{\text{reduced}})$ is found, following the formula $W(\text{token}|\text{context}) = B(\text{context}) + W(\text{token}|\text{context}_1)$. If the start state is reached (empty context), and *token* is not found, then $\langle unk \rangle$ weight is assigned.

Graph Representation. In practice, n-gram LM is represented as a graph, specifically a suffix tree. Prefixes (forward state chains) represent n-grams. *States* represent context. *Suffix links* represent fallbacks to a shorter context (backoff transitions). Fig. 1 visualizes such representation for 3-gram word LM built from the sentence “the cat sat on the mat”. Note that for 3-gram transitions, the arcs point directly to 2-gram states since no separate states are needed for the highest-order n-grams. To simplify storage, models often omit explicit representation of the $\langle s \rangle$ and $\langle /s \rangle$ tokens. Instead, the start-of-sequence is treated as a separate state, while the end-of-sequence $\langle /s \rangle$ acts as a final weight marker (double-border

Algorithm 1 Inference of NGPU-LM

Require: state - current LM state for hypothesis

lm_params (tensors):

arcs: arc_tokens, arc_weights, arc_to_states

states to arcs ranges: start_arcs, end_arcs

states to backoffs: boff_weights, boff_to_states

```

1: next_states = full([V], fill_value = -1)
2: next_scores = empty([V])
3: acc_boff = 0.0 # backoff weight accumulator
4: while (next_states == -1).any() do
5:   scores = zeros([V])
6:   to_states = full([V], fill_value = -1)
7:   # get arcs for the current state
8:   s, e = start_arcs[state], end_arcs[state]
9:   tokens = arc_tokens[s : e]
10:  scores[tokens] = acc_boff + arc_weights[s : e]
11:  to_states[tokens] = arc_to_states[s : e]
12:  # fill in data for not yet observed tokens
13:  mask = (next_states == -1) # not visited states mask
14:  next_scores[mask] = scores[mask]
15:  next_states[mask] = to_states[mask]
16:  # traverse by backoff transition
17:  acc_boff += boff_weights[state]
18:  state = boff_to_states[state]
19: end while
20: return next_scores, next_states

```

circle in the figure).

Standard graph implementations, such as CPU-focused libraries like KenLM [17] and parallel computing toolkits like gLM [21], are designed to efficiently handle queries of the form “(token, state) $\rightarrow$ (token_weight, next_state)”. To achieve high performance, queries $P(\text{token}|\text{context})$ are typically processed in one of two ways [17], using sorted transition arrays to enable binary search (or K-ary for parallel implementations), or by employing hash tables for $O(1)$ transition lookups.

However, when probabilities need to be aggregated across the entire vocabulary (we use BPE [22] tokens in our experiments), performing separate queries for each token is computationally expensive. Instead, we can determine the range of transitions to a given state and fill in the probabilities for tokens present in that state in parallel. Secondly, since the length of backoff chains is limited by the order of the LM (up to 10 in our experiments), the number of iterations required to aggregate probabilities over the full vocabulary remains relatively small. These two insights enable the construction of a simple yet highly efficient data structure and parallelized algorithm for full-vocabulary batched queries.

2.2. NGPU-LM Data Structure.

For efficient greedy decoding with full vocabulary scoring, we need to support queries like $\text{state}[B] \rightarrow (\text{token_weights}[B, V], \text{next_states}[B, V])$, where the input is a batch of states (representing a batch of hypotheses), and the output is the probability distribution produced by LM for the whole vocabulary $|V|$ for each of state in the batch, along with the corresponding next states. To achieve this, we store all the data in tensors, specifically:

- *Arcs (transitions) data tensors:* tokens, weights, source states (from_states), target states (to_states)
- *State data tensors:* backoff target states (boff_to_states), backoff weights, and final weights.

For the start (unigram) state, we store transitions for all vocabulary tokens to avoid extra checks during queries. If the token is absent in ARPA, we use normalized $\langle unk \rangle$ weight,

eliminating the need for a separate unknown token representation. This ensures that retrieving probabilities for zero-context queries (unigrams) is straightforward.

To optimize retrieval, we sort all arcs by the tuple “(from_state, token)”, and introduce two additional tensors, `start_arcs` and `end_arcs`, which define the range of arcs for each state. This allows for efficient weight retrieval using a simple parallelized “index-select” operation. Specifically, for non-batched queries for state s , we first get (start, end) transitions boundaries and then assign scores and next states. To obtain scores over the full vocabulary, the algorithm iteratively follows backoff transitions, aggregating the weights for present tokens until all scores are found. The maximum number of iterations does not exceed the order of the model. The full non-batched algorithm is listed in Algorithm 1. It can be easily extended to support the input of the batch of states. We are omitting the batched version here due to space limits and encourage the readers to check the complete implementation¹.

Implementation and Acceleration. The baseline algorithm is implemented in PyTorch. However, loop conditions are evaluated on CPU (due to eager mode constraints), slowing down execution. This fact also makes the implementation incompatible with CUDA graphs, limiting decoding speed optimizations. To overcome these issues, we implemented a custom kernel in Triton [23], which retains the algorithm’s clarity while significantly improving speed. Our PyTorch implementation remains available as a fallback option for environments where Triton cannot be used (e.g., CPU-based decoding).

2.3. Greedy Decoding with NGPU-LM

In typical implementations of the beam search decoding [24, 20] with LM, to extend the hypothesis, the algorithm first selects top-k token expansions for the hypothesis and then the LM scores these tokens, retrieving respecting weights one by one. This approach is not applicable for greedy decoding since, on each step, only one expansion is selected, and LM is not useful. To enable the use of LM in greedy mode, we need to use full vocabulary scoring, first adding language model weight to the expanded hypotheses in each step and then applying greedy selection. Such approach would be impractically slow with conventional LM implementations due to CPU-bound sequential queries but is efficient with NGPU-LM.

Transducers. We integrate NGPU-LM into the decoding pipeline for conventional transducer (RNN-T) [3] and the Token-and-Duration Transducer (TDT) [25]. Our implementation is built on the greedy label-looping [18] algorithm accelerated with CUDA graphs [19] to ensure maximum decoding efficiency. In initial experiments, we found that directly combining transducer and language model scores led to an increased deletion rate. This issue arises due to the dominance of the blank symbol (which is not scored by LM), negatively impacting performance. To address this, we introduce a two-stage token selection algorithm that maintains a distinction between blank and non-blank predictions at each decoding step. Firstly, we do standard greedy token prediction. If the $\langle blank \rangle$ symbol is predicted, we retain it. Otherwise, we rescore all non-blank probabilities using the language model and apply greedy selection among non-blank symbols. This approach allows for an increase in language model weight effectively, leading to significant performance improvements, especially for out-of-domain.

CTC. For CTC model decoding, we adopt a similar ap-

proach to transducer-based decoding but introduce an additional distinction between symbol categories at each step. Specifically, we preserve three groups: “blank”, “repeated (previous) token” and “other tokens”. Since repeated tokens are collapsed due to the CTC decoding rule, they should not be rescored by the language model. To accelerate sequential autoregressive processing with LM, which involves tiny operations with significant kernel launch overhead, we apply CUDA graphs similar to the approach used for transducers [19].

Attention Encoder-Decoder. Unlike transducer and CTC models, encoder-decoder models with attention do not use the “blank” symbol. Instead, they introduce an end-of-sentence “ $\langle eos \rangle$ ” symbol, which directly corresponds to the final weight in the language model. The final weight is either directly listed in n-grams or is accessible by backoff transitions. To optimize final weight retrieval during decoding, we precompute and store final weights for all states at the model loading time by traversing backoff transitions. The preprocessing step significantly simplifies and accelerates the decoding. We integrate NGPU-LM into both greedy and beam search decoding.

3. Experimental Setup

3.1. ASR modeling

We evaluated four ASR architectures: CTC, Transducers (RNN-T and TDT), and AED. For all models, we employed the FastConformer encoder [26, 27], configured with 108M parameters. The RNN-T and TDT models utilize a single-layer LSTM decoder with 640 hidden units, resulting in 114M parameters in total, while the AED model features a 4-layer Transformer decoder with a hidden size of 1024 (179M total model params).

To simulate varying resource conditions, we experimented with two training datasets: a large corpus of approximately 24k hours of English speech from various open sources and the 960-hour Librispeech dataset [28], representing a low-resource scenario. Text data was tokenized using a BPE tokenizer [22] with a vocabulary of 1024 tokens. All models were trained under identical conditions using the NeMo [20] framework.

To assess model performance, we tested on out-of-domain datasets: SLURP (command-focused) [29] and SPGISpeech (finance-related) [30] (SPGI). We built² token-level 10-gram language models (without pruning, except for SPGI³) using KenLM toolkit [17] for these domains using their respective training data. The development sets were then used to fine-tune the fusion weights for the external language models. All models require less than 100MB on the GPU memory, with the exception of the model built on SPGI, which uses 1.5GB. For the ASR model trained on LibriSpeech data, we additionally evaluate its performance on the WSJ `eval92` dataset [31], using `dev93` to tune the language model weight. For beam search with transducers, we use Modified Adaptive Expansion Search [32] with KenLM [17] model fusion. We use Flashlight [33] decoder for CTC models. In all beam search runs, $beam = 4$ is used.

Factorized and stateless Transducers. There are a number of studies that have explored factorized transducers [34, 35, 36], where the primary idea is to decouple label probabilities from the blank token. This separation facilitates the estimation of the internal language model (ILM) and its subtraction during the shallow fusion with an external LM. In our work, we investigate the integration of the proposed NGPU-LM for Hybrid Autoregressive Transducer (HAT) [37], using the same

¹<https://github.com/NVIDIA/NeMo/pull/10989>

²github.com/NVIDIA/NeMo/blob/main/train_kenlm.py

³`ngram.prune=[0,0,0,0,1,1,2,2,3,3]`

Table 1: *Decoding Fast Conformer Large models (108M Encoder parameters) trained on large data (24k hours) on out-of-domain data (SLURP, SPGI). WER vs RTFx. RTFx is the inversed real-time factor (higher is better).*

Decoding	LM	SLURP		SPGI	
		WER %↓	RTF↑	WER %↓	RTF↑
RNN-T					
greedy	-	22.47	996	5.93	2352
greedy	NGPU-LM	20.36	973	5.43	2230
beam	-	22.34	87	6.26	95
beam	LM	19.34	79	4.73	87
TDT					
greedy	-	23.48	1076	6.45	2877
greedy	NGPU-LM	21.21	1063	5.91	2798
beam	-	22.99	95	6.29	101
beam	LM	19.48	83	4.89	88
AED					
greedy	-	25.06	732	6.10	785
greedy	NGPU-LM	23.59	722	5.48	736
beam	-	24.31	244	6.08	292
beam	NGPU-LM	19.68	442	4.65	286
CTC					
greedy	-	24.88	1175	6.75	3311
greedy	NGPU-LM	22.66	1152	6.34	3108
beam	-	24.88	941	6.75	1905
beam	LM	20.32	838	4.42	1294

Table 2: *Decoding Fast Conformer Large models (108M Encoder parameters) trained on LibriSpeech dataset (960h). WER %. Test data: SLURP, SPGI, WSJ.*

Model	Decoding	LM	SLURP	SPGI	WSJ
RNN-T	greedy	-	48.66	14.65	7.25
	greedy	NGPU-LM	44.90	13.10	6.36
	beam	LM	44.53	12.88	6.75
CTC	greedy	-	51.06	16.09	7.71
	greedy	NGPU-LM	47.58	14.41	6.47
	beam	LM	42.67	11.51	5.14

training dataset of 24k hours. Additionally, we examine a stateless decoder [38] that replaces the conventional LSTM-based decoder with embeddings of the last two predicted tokens. The lightweight decoder may be less biased toward the training data, making it more suitable for shallow fusion with an external LM.

Metrics. We measure decoding accuracy using the Word Error Rate (WER) metric and assess decoding speed with the inverse Real-Time Factor (RTFx). RTFx is calculated as the total audio duration divided by the evaluation time of the ASR model. RTFx is measured after a single warm-up run on a single A6000 GPU with a batch size of 32.

4. Results

Table 1 presents the decoding performance of models trained on sufficiently large datasets. The results demonstrate that integrating NGPU-LM into greedy decoding significantly improves accuracy while incurring less than 7% computational overhead across all models.

RNN-T and TDT Models. With the proposed greedy decoding using NGPU-LM, we achieve relative word error rate reduction (WERR) ranging from 8.4% (both models on SPGI) to 9.7% (TDT on SLURP). Notably, beam search without an LM fails to provide a significant accuracy improvement while causing a substantial 11x performance degradation. This indicates that the LM itself is not a bottleneck in beam search. On

Table 3: *Comparison of RNN-T and HAT models with LSTM and Stateless decoders depending on language model setup.*

Model	Decoder	LM Setup	WER %		
			SLURP	SPGI	Internal
RNN-T	LSTM	- +LM	22.47 20.36	5.93 5.43	11.81 9.84
	Stateless	- +LM	22.90 20.91	5.86 5.41	11.90 10.23
HAT	LSTM	- +LM -ILM+LM	22.96 20.82 19.79	5.99 5.42 5.47	12.41 10.17 10.12
		- +LM -ILM+LM	23.14 21.20 20.42	5.97 5.43 5.50	12.45 10.46 10.19

SLURP, our greedy decoding approach accounts for over 56% of the relative improvement provided by beam search and more than 30% on SPGI.

AED. For the AED model, we observe WERR improvements between 5.9% (SLURP) and 10.2% (SPGI) when using NGPU-LM in greedy mode. Interestingly, when applying beam search without an LM on SLURP, we notice abnormal performance degradation due to the model’s inability to generate the end-of-sequence symbol. This issue is resolved by LM fusion.

CTC models. CTC models also benefit from NGPU-LM, achieving 6.1% to 8.9% WERR improvements. However, the performance gap between the proposed greedy decoding and beam search is the largest among all model types.

Low-Resource Scenario. Table 2 presents results for models trained on smaller data, showing trends similar to high-resource cases. In this setting, the proposed approach recovers more than 87% of the gap between greedy and beam search for the RNN-T model with up to 10.6% WERR improvement on SLURP and SPGI, and outperforms beam search on WSJ data. For the CTC model, it covers approximately 35% of the gap.

Factorized and stateless Transducers. The comparison results of RNN-T and HAT models with different decoders are presented in Table 3. HAT_{LSTM} showed the best result on SLURP, but slightly inferior to RNN-T_{LSTM} in the case of SPGI. The use of a stateless decoder did not show a clear advantage of this approach in the conditions of LM fusion. In addition, we evaluated our internal dataset designed to test keywords recognition in the computer science domain. The language model for this set was built on the list of target keywords consisting of 200 elements. The obtained results showed the effectiveness of the proposed method in the conditions of keyword boosting with the best result for the RNN-T_{LSTM} model.

5. Conclusion

We introduced a novel highly parallelized representation of n-gram language models, enabling fast, batched, full-vocabulary queries. This approach seamlessly integrates into any ASR decoding pipeline. Additionally, we introduce efficient language model fusion for greedy decoding for all types of ASR models with minimal computational overhead. Our results demonstrate that this customization significantly improves out-of-domain performance demonstrating up to 10.6% WERR improvement while avoiding the high computational cost of beam search, making it particularly valuable for industry-scale ASR systems. In future work, we plan to explore deeper integration of our model into various beam search algorithms, further enhancing decoding efficiency and accuracy.

6. References

- [1] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks,” in *ICML*, 2006.
- [2] G. E. Hinton, L. Deng, D. Yu, G. E. Dahl, A. rahman Mohamed, N. Jaitly, A. W. Senior, V. Vanhoucke, P. Nguyen, T. N. Sainath, and B. Kingsbury, “Deep neural networks for acoustic modeling in speech recognition,” *IEEE Signal Processing Magazine*, vol. 29, p. 82, 2012.
- [3] A. Graves, “Sequence transduction with recurrent neural networks,” in *ICML*, 2012.
- [4] W. Chan, N. Jaitly, Q. V. Le, and O. Vinyals, “Listen, attend and spell: A neural network for large vocabulary conversational speech recognition,” *ICASSP*, 2016.
- [5] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *ICML*, 2022.
- [6] K. C. Puvvada, P. Želasko, H. Huang *et al.*, “Less is more: Accurate speech recognition & translation without web-scale data,” in *Interspeech*, 2024.
- [7] D. Zhang, S. Li, X. Zhang, J. Zhan, P. Wang, Y. Zhou, and X. Qiu, “Speechgpt: Empowering large language models with intrinsic cross-modal conversational abilities,” in *Conference on Empirical Methods in Natural Language Processing*, 2023.
- [8] P. K. Rubenstein, C. Asawaroengchai, D. D. Nguyen *et al.*, “Audiodiopalm: A large language model that can speak and listen,” *ArXiv*, vol. abs/2306.12925, 2023.
- [9] G. Pundak, T. N. Sainath, R. Prabhavalkar, A. Kannan, and D. Zhao, “Deep context: End-to-end contextual speech recognition,” in *SLT*, 20218.
- [10] M. Jain, G. Keren, J. Mahadeokar, and Y. Saraf, “Contextual rnn-t for open domain asr,” in *Interspeech*, 2020.
- [11] Z. Chen, H. Huang, A. Andrusenko, O. Hrinchuk, K. C. Puvvada, J. Li, S. Ghosh, J. Balam, and B. Ginsburg, “Salm: Speech-augmented language model with in-context learning for speech recognition and translation,” in *ICASSP 2024*. IEEE, 2024.
- [12] D. Zhao, T. N. Sainath, D. Rybach, P. Rondon, D. Bhatia, B. Li, and R. Pang, “Shallow-fusion end-to-end contextual biasing,” in *Interspeech*, 2019.
- [13] N. Jung, G. min Kim, and J. S. Chung, “Spell my name: Keyword boosted speech recognition,” in *ICASSP*, 2022.
- [14] A. Andrusenko, A. Laptev, V. Bataev, V. Lavrukhin, and B. Ginsburg, “Fast context-biasing for ctc and transducer asr models with ctc-based word spotter,” *Interspeech*, 2024.
- [15] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, pp. 1735–1780, 1997.
- [16] A. Vaswani, N. M. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Neural Information Processing Systems*, 2017.
- [17] K. Heafield, “KenLM: Faster and smaller language model queries,” in *Proceedings of the Sixth Workshop on Statistical Machine Translation*. Edinburgh, Scotland: Association for Computational Linguistics, 2011, pp. 187–197.
- [18] V. Bataev, H. Xu, D. Galvez, V. Lavrukhin, and B. Ginsburg, “Label-looping: Highly efficient decoding for transducers,” in *2024 IEEE Spoken Language Technology Workshop (SLT)*, 2024, pp. 7–13.
- [19] D. Galvez, V. Bataev, H. Xu, and T. Kaldewey, “Speed of light exact greedy decoding for rnn-t speech recognition models on gpu,” in *Interspeech 2024*, 2024, pp. 277–281.
- [20] O. Kuchaiev, J. Li, H. Nguyen *et al.*, “Nemo: a toolkit for building ai applications using neural modules,” in *NeurIPS Workshop on Systems for ML*, 2019.
- [21] N. Bogoychev and A. Lopez, “N-gram language models for massively parallel devices,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2016, pp. 1944–1953.
- [22] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 2016.
- [23] P. Tillet, H.-T. Kung, and D. Cox, “Triton: an intermediate language and compiler for tiled neural network computations,” in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019, pp. 10–19.
- [24] S. Watanabe, T. Hori, S. Karita, T. Hayashi, J. Nishitoba *et al.*, “ESPnet: End-to-end speech processing toolkit,” in *Interspeech*, 2018.
- [25] H. Xu, F. Jia, S. Majumdar, H. Huang, S. Watanabe, and B. Ginsburg, “Efficient sequence transduction by jointly predicting tokens and durations,” in *ICML*, 2023.
- [26] A. Gulati, J. Qin, C.-C. Chiu *et al.*, “Conformer: Convolution-augmented Transformer for speech recognition,” in *Interspeech*, 2020.
- [27] D. Rekesh, N. R. Koluguri, S. Krizan, *et al.*, “Fast Conformer with linearly scalable attention for efficient speech recognition,” in *Automatic Speech Recognition and Understanding Workshop (ASRU)*, 2023.
- [28] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “Librispeech: An asr corpus based on public domain audio books,” *ICASSP*, 2015.
- [29] E. Bastianelli, A. Vanzo, P. Swietojanski, and V. Rieser, “SLURP: A spoken language understanding resource package,” in *EMNLP*, 2020.
- [30] P. K. O’Neill, V. Lavrukhin, S. Majumdar *et al.*, “Spgispeech: 5, 000 hours of transcribed financial audio for fully formatted end-to-end speech recognition,” in *Interspeech*, 2021.
- [31] D. B. Paul and J. Baker, “The design for the Wall Street Journal-based CSR corpus,” in *Speech and Natural Language Workshop*, 1992.
- [32] J. Kim, Y. Lee, and E. Kim, “Accelerating RNN Transducer inference via adaptive expansion search,” *IEEE Signal Processing Letters*, 2020.
- [33] J. Kahn, V. Pratap, T. Likhomanenko *et al.*, “Flashlight: Enabling innovation in tools for machine learning,” 2022.
- [34] E. McDermott, H. Sak, and E. Variani, “A density ratio approach to language model fusion in end-to-end automatic speech recognition,” *ASRU*, 2019.
- [35] Z. Meng, S. Parthasarathy, E. Sun, and other, “Internal language model estimation for domain-adaptive end-to-end speech recognition,” *SLT*, 2021.
- [36] Z. Meng, T. Chen, R. Prabhavalkar, and other, “Modular hybrid autoregressive transducer,” *SLT*, 2022.
- [37] E. Variani, D. Rybach, C. Allauzen, and M. Riley, “Hybrid autoregressive transducer (hat),” *ICASSP*, 2020.
- [38] M. R. Ghodsi, X. Liu, J. A. Apfel, R. Cabrera, and E. Weinstein, “RNN-Transducer with stateless prediction network,” *ICASSP*, 2020.